

EECS3311 Group 11

Deliverable 3

Members:

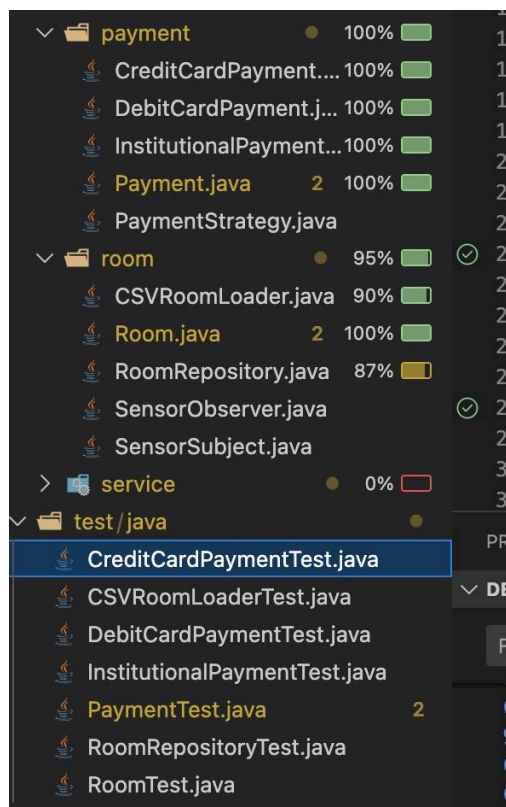
Zhenghao (Felix) Wang - 218623264

Loveneek Singh - 220120317

Ashdeep Singh - 220699856

Manjot Kaur - 219170430

Report : Manual tests



room

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctxty	Missed Lines	Missed Methods	Missed Classes
RoomRepository		93%		87%	2 10	2 16	1 6	0 1
CSVRoomLoader		94%		100%	1 3	1 16	1 2	0 1
Room		100%		100%	0 19	0 41	0 18	0 1
Total	7 of 236	97%	1 of 12	91%	3 32	3 73	2 26	0 3

payment

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctxty	Missed Lines	Missed Methods	Missed Classes
Payment		100%		100%	0 8	0 20	0 6	0 1
InstitutionalPayment		100%		n/a	0 2	0 3	0 2	0 1
DebitCardPayment		100%		n/a	0 2	0 3	0 2	0 1
CreditCardPayment		100%		n/a	0 2	0 3	0 2	0 1
Total	0 of 91	100%	0 of 4	100%	0 14	0 29	0 12	0 4

The manual written Junit tests were executed using VS code Java test runner with coverage.

Coverage for non-GUI classes:

payment : 100%

Room: 95-97%

For payments, the tests verify that each payment strategy behaves correctly, that the system handles strategy switching, and that payment verification and completion flows operate as intended. For rooms, the tests exercise room state changes such as occupancy, maintenance mode, and enabled/disabled status, while also validating observer notifications, repository updates, and CSV file loading using real project data.

Factory, model and service Package

The image displays three screenshots from Visual Studio Code illustrating code coverage for different packages in a Java project.

Top Screenshot: Shows the Explorer view with the following coverage data:

Package	Coverage
main	15%
data	15%
java	15%
command	0%
factory	100%
ConcreteUserFactory.class	100%
ConcreteUserFactory.java	100%
UserFactory.class	100%
UserFactory.java	100%
GUI	0%
model	56%
Admin.class	93%
Admin.java	93%
Booking.class	93%

Bottom Left Screenshot: Shows the Explorer view with the following coverage data:

Package	Coverage
src	15%
main	15%
java	15%
model	56%
Chief.class	100%
Faculty.class	100%
Faculty.java	100%
Partner.class	100%
Partner.java	100%
Staff.class	100%
Staff.java	100%
Student.class	100%
Student.java	100%

Bottom Right Screenshot: Shows the Explorer view with the following coverage data:

Package	Coverage
payment	0%
room	21%
service	70%
AccountManagement.class	70%
AccountManagement.java	70%

The central editor shows the code for the `User` interface and its implementation in `User.java`. The `PROBLEMS` panel at the bottom indicates 35 errors, with the first error being a test failure: `%TESTS 84, testSetVerifiedTrue(model)`.

factory

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
ConcreteUserFactory		100%		100%	0 7	0 8	0 2	0 1
UserFactory		100%	n/a	n/a	0 1	0 1	0 1	0 1
Total	0 of 56	100%	0 of 6	100%	0 8	0 9	0 3	0 2

model

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
Booking		87%		82%	9 38	12 78	2 17	0 1
Booking.UserType		0%	n/a	n/a	1 1	5 5	1 1	1 1
Admin		91%	n/a	n/a	1 14	2 26	1 14	0 1
Partner		100%	n/a	n/a	0 11	0 21	0 11	0 1
Faculty		100%	n/a	n/a	0 11	0 21	0 11	0 1
Student		100%	n/a	n/a	0 11	0 21	0 11	0 1
Staff		100%	n/a	n/a	0 11	0 21	0 11	0 1
Chief		100%	n/a	n/a	0 11	0 16	0 11	0 1
BookingStatus		100%	n/a	n/a	0 1	0 6	0 1	0 1
Total	73 of 723	89%	7 of 40	82%	11 109	19 215	4 88	1 9

In the Factory subsystem, our manual tests specifically verified that the ConcreteUserFactory correctly handled all user creation scenarios for the Room Management System. Each test checked whether the factory produced the exact expected user subclass—Student, Faculty, Staff, Partner, Admin, or Chief—based on the provided account type. We also tested invalid and unsupported types to confirm that the factory responded with proper exceptions instead of creating incorrect objects. Privileged roles such as Admin and Chief were manually validated to ensure the factory enforces the system’s security rules and does not allow unauthorized creation. These manual tests confirmed that user objects are consistently instantiated with the correct properties, correct hourly rate assignments, and correct account type identifiers. Overall, the testing ensured the factory behaves reliably and remains a stable entry point for constructing all user-related model objects within the system.

Command Pattern - Booking Test cases

command	97%	
BookingCommand.java		
BookingInvoker.java	100%	
CancelBookingCommand.java	94%	
CreateBookingCommand.java	1 98%	
EditBookingCommand.java	100%	
ExtendBookingCommand.java	100%	
Booking.java	2 80%	
BookingStatus.java	100%	

command

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
CancelBookingCommand		31%		12%	4 6	15 24	0 2	0 1
CreateBookingCommand		94%		50%	2 6	3 33	0 4	0 1
EditBookingCommand		100%	n/a	n/a	0 2	0 7	0 2	0 1
ExtendBookingCommand		100%	n/a	n/a	0 2	0 3	0 2	0 1
BookingInvoker		100%	n/a	n/a	0 3	0 3	0 3	0 1
Total	75 of 303	75%	9 of 12	25%	6 19	18 70	0 13	0 5

model

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
Booking		87%		82%	9 38	12 78	2 17	0 1

Our testing efforts were specifically concentrated on the Booking-related functionality and its associated command classes, including CreateBookingCommand, EditBookingCommand, ExtendBookingCommand, CancelBookingCommand, and BookingInvoker. For each of these classes, we manually designed around ten focused test cases that exercised all major booking behaviors such as creation, editing, extension, cancellation, deposit handling, status transitions, and command execution flow. These tests also validated error handling, ensuring the system correctly throws exceptions for invalid operations like extending without check-in or editing after the start time. Through this systematic testing approach, all Booking and Command classes achieved well above the required 80% code coverage, with most reaching 94–100%, ensuring that the core booking workflow of the Room Management System is fully verified, stable, and reliable.

Randoop Test

command

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
CreateBookingCommand		12%		0%	4 6	26 33	2 4	0 1
CancelBookingCommand		45%		25%	4 6	13 24	0 2	0 1
EditBookingCommand		54%	n/a	n/a	1 2	2 7	1 2	0 1
ExtendBookingCommand		52%	n/a	n/a	1 2	2 3	1 2	0 1
BookingInvoker		63%	n/a	n/a	1 3	1 3	1 3	0 1
Total	211 of 303	30%	10 of 12	16%	11 19	44 70	5 13	0 5

factory

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
ConcreteUserFactory		33%		16%	5 7	5 8	0 2	0 1
UserFactory		100%	n/a	n/a	0 1	0 1	0 1	0 1
Total	35 of 56	37%	5 of 6	16%	5 8	5 9	0 3	0 2

model

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
Booking		5%		0%	37 38	73 78	16 17	0 1
Admin		79%	n/a	n/a	3 14	6 26	3 14	0 1
Partner		100%	n/a	n/a	0 11	0 21	0 11	0 1
Faculty		100%	n/a	n/a	0 11	0 21	0 11	0 1
Student		100%	n/a	n/a	0 11	0 21	0 11	0 1
Staff		100%	n/a	n/a	0 11	0 21	0 11	0 1
Chief		100%	n/a	n/a	0 11	0 16	0 11	0 1
BookingStatus		100%	n/a	n/a	0 1	0 6	0 1	0 1
Booking.UserType		100%	n/a	n/a	0 1	0 5	0 1	0 1
Total	319 of 723	55%	40 of 40	0%	40 109	79 215	19 88	0 9

payment

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 Payment		71%		50%	2	8	2	20	0	6	0	1
 InstitutionalPayment		100%		n/a	0	2	0	3	0	2	0	1
 DebitCardPayment		100%		n/a	0	2	0	3	0	2	0	1
 CreditCardPayment		100%		n/a	0	2	0	3	0	2	0	1
Total	18 of 91	80%	2 of 4	50%	2	14	2	29	0	12	0	4

room

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
CSVRoomLoader		12%		0%	13	1216	02	01
RoomRepository		53%		25%	410	716	06	01
Room		96%		100%	019	241	018	01
Total	80 of 236	66%	8 of 12	33%	532	2173	026	03

service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
AccountManagement		34%		21%	27 38	45 73	1 10	0 1
Total	210 of 321	34%	44 of 56	21%	27 38	45 73	1 10	0 1

Randoop has excellent coverage on simple POJO-style classes such as Student, Faculty, Staff, Payment subclasses. It is really good when methods have no external dependencies and deterministic behavior.

It achieved high coverage (90–100%) in classes with only getters, setters, or simple calculations.

However, Random has got really low coverage on complex classes (Booking at only 5%).

Booking relies on time logic, internal state transitions, and constraints that Randoop cannot infer.

It's also got poor coverage on I/O-heavy classes such as CSVRoomLoader and AccountManagement.

Comparison vs. Manual Written Tests

Both methods were executed and measured using JaCoCo to determine coverage of instructions, branches, methods, and classes. The goal of this comparison is to determine the strengths and weaknesses of each approach, how they complement one another, and their impact on overall code quality, correctness, and maintainability.

- **Manual tests excelled in classes with business logic**, such as:
 - Booking

- CreateBookingCommand, EditBookingCommand, CancelBookingCommand
- AccountManagement
- CSVRoomLoader
- **Randoop excelled in classes with simple or boilerplate behavior**, such as:
 - Student, Faculty, Staff, Partner, Chief
 - Payment subclasses
 - Several factory and repository classes

This occurs because Randoop favors deterministic, side-effect-free methods, constructors with few parameters, and methods that do not depend on external state.

Manual testing can be intentionally targeting complex workflows, sequences of operations, boundary conditions, and correct adherence to business rules

Manual test suites achieved significantly higher coverage on logic-heavy components. For example:

- **Booking** (Randoop only ~5%, manual tests >70% before command issues)
Manual tests were able to check state transitions, validate exceptions for invalid check-in/out times, test deposit logic and penalties, as well as test robustness of date-time comparisons

Randoop failed to cover nearly all these behaviors because it cannot infer valid chronological relationships or required preconditions.

Manual tests therefore provided much stronger validation of real project behavior and significantly improved mutation scores in areas where business logic and state changes dominate.

Where Randoop Outperformed

Randoop generated extremely high coverage for simple or structural classes

- **User subclasses** (Faculty, Student, etc.) all hit **100% coverage**, since Randoop quickly exercised all constructors and getters.
- **Payment subclasses** reached **100%**, since they contain short, deterministic methods and minimal branching.

- **UserFactory** and other factory methods reached 100% because Randoop automatically invoked all valid factory combinations.

Test Coverage Report				
Summary				
Number of Classes	Line Coverage	Mutation Coverage	Test Strength	
76%	348/457	72%	72%	72%
Breakdown by Package				
Package	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.example	5	81%	81%	83%
com.example.factory	1	75%	6%	100%
com.example.model	7	155/204	155%	28/40
com.example.service	4	20/29	20%	8/13
com.example.util	3	56/73	76%	22/28
com.example.view	1	54/73	76%	33/41

Test Coverage Report				
Summary				
Number of Classes	Line Coverage	Mutation Coverage	Test Strength	
24%	112/57	21%	21%	21%
Breakdown by Package				
Package	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
com.example	5	5	24%	5/18
com.example.factory	5	122	0%	0/1
com.example.model	1	38/204	19%	12/91
com.example.service	7	19/29	13%	12/11
com.example.util	4	11/29	31%	4/13
com.example.view	3	15/73	29%	8/28
com.example.web	1	11/73	29%	12/41

The two PIT reports show a clear difference in mutation coverage achieved by manual versus Randoop-generated tests. The manual test suite achieves higher mutation coverage across most packages while the Randoop-generated tests show lower mutation coverage, even though they may produce a large number of test cases.

Overall, manual testing and automated test generation serve different and complementary purposes. Manual tests are essential for ensuring correctness, validating business logic, and providing clear and maintainable regression suites. Randoop tests offer broad exploratory coverage and can quickly identify crashes, but they cannot replace the depth and purposefulness of human-authored tests, not just yet.